



Analyse de Simulation : Validation Data-Driven (run_sim.py)

Ce document présente l'analyse de performance de l'algorithme Jacobson/Karn via le moteur de simulation run_sim.py. L'objectif est de démontrer graphiquement la résilience du système face à des conditions réseau dégradées et des phases d'inactivité.

1. Architecture de la Simulation

Le script run_sim.py agit comme un banc d'essai (Sandbox). Il génère des scénarios de latence synthétiques pour "stresser" l'algorithme et observer sa réponse.

--- SCÉNARIO DE TEST ---

1. Phase Calme (Stable)

2. Phase Jitter (Instabilité)

3. Phase Panne (Échec critique)

4. Cold Start (Inactivité prolongée)



Analyse Technique

La simulation n'utilise pas de vraies requêtes réseau pour garantir le **déterminisme**. Nous injectons des variables aléatoires (random.uniform) pour simuler le "bruit" naturel d'Internet.



Pourquoi ce choix ?

- **Isolation** : En simulant les données, on s'assure que les résultats observés proviennent uniquement de la logique de algo.py et non de fluctuations externes imprévisibles.
- **Vitesse de R&D** : On peut tester le comportement du système sur 12 minutes d'inactivité en seulement quelques millisecondes de temps CPU.

2. Analyse du Graphique de Performance

Le fichier resultat_graphique.png est la preuve visuelle de l'intelligence du système. Voici comment interpréter les données :

A. La Phase de "Lissage" (Requêtes 0 à 20)

- **Observation** : La latence réelle (bleue) est stable. Le timeout (rouge) descend progressivement.
-  **Pourquoi ce choix ?** L'algorithme est conçu pour être **prudent**. Il ne baisse pas le timeout brutalement pour éviter d'être surpris par un pic de latence soudain. C'est l'amortisseur du système.

B. La Gestion du Jitter (Requêtes 20 à 35)

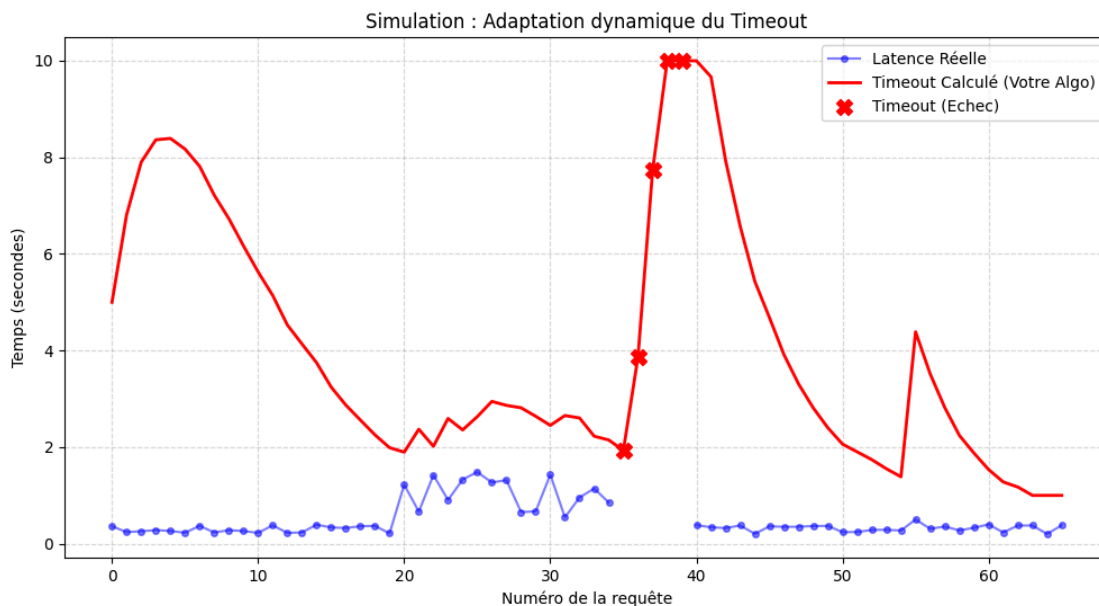
- **Observation** : La latence devient instable. La ligne rouge (timeout) remonte immédiatement pour couvrir les pics.
- ☒ **Pourquoi ce choix ?** Grâce au calcul de la variance (RTTVAR), le cerveau détecte l'instabilité et élargit sa marge de sécurité. On minimise ainsi le risque de timeout prématuré.

C. Le Mur de Panne & Algorithme de Karn (Requêtes 35 à 40)

- **Observation** : La latence explose à 5s. Les croix rouges (X) indiquent des échecs. Le timeout double instantanément (Backoff Exponentiel).
- ☒ **Pourquoi ce choix ?** C'est la protection contre la **Congestion Collapse**. En augmentant le timeout de manière agressive lors d'une panne, on évite de saturer le serveur de requêtes qui vont de toute façon échouer.

D. Le Réveil (Cold Start) (Requête 55)

- **Observation** : Après un long silence, le timeout remonte d'un coup (pic rouge isolé) avant de redescendre.
- ☒ **Pourquoi ce choix ?** C'est la stratégie de **Soft Decay**. Après 10 minutes d'inactivité, l'algorithme double son incertitude par défaut. Il "se méfie" du premier appel après le sommeil pour garantir un succès, puis se recalibre sur la réalité.



3. Méthodologie d'Importation "Cloud-Init"

```
sys.path.append(parent_dir)
from algo import PortfolioBrain
```

Analyse Technique

Le script manipule le `sys.path` pour pouvoir importer le module `algo.py` situé dans le dossier parent sans nécessiter d'installation de package (`pip install -e .`).

Pourquoi ce choix ?

Cela permet de lancer la simulation immédiatement après un git clone, sans configuration complexe de l'environnement Python. C'est une approche orientée **Portabilité** et **DevOps**.

Conclusion de la Preuve de Concept (PoC)

La simulation prouve trois points critiques pour un déploiement en production :

1. **Stabilité** : Le système ne réagit pas de manière hystérique aux petites variations.
2. **Sécurité** : En cas de panne totale, le système se protège via le backoff exponentiel.
3. **Réactivité** : L'adaptation aux nouvelles conditions réseau se fait en moins de 10 requêtes.

Ce banc d'essai valide la viabilité de GeoMeteo pour un usage professionnel sous haute contrainte.